

Full-Stack Engineering Code Challenge: Commission Quote App

Welcome, and thank you for interviewing with us!

To get a better sense of your coding style, decision-making, and problem-solving approach, we would like you to complete a short take-home challenge. We know your time is valuable, so we have intentionally kept the scope narrow.

Timebox Guideline

Please spend no more than 4 hours on this. We are not looking for a production-ready application. We want to see readable code, a working solution, basic tests, clear edge-case handling, and simple instructions for running it.

Context

In our Lending Platform, staff members frequently need to generate "Commission Quotes" based on loan applications. To do this, our system needs to process loan details and send them to an external Vendor system, which calculates the commission and returns a quote.

Currently, the external vendor Commission Quote API is **under construction and not yet available**. However, the API contract and authentication requirements have been finalized. We need to unblock our development by building our application against a mock version of this API.

The Task & Requirements

Your task is to build a web application that allows a user to input loan details and displays the generated commission quote, along with a mock version of this Commission Quote API that simulates the external vendor.

1. Web Application Requirements

- A user interface with a form to capture basic loan details (e.g., `loanAmount`, `loanTermInMonths`, `riskBand`).
- A "Generate Quote" button.
- A display area to show the resulting quote data when the request is successful.
- Proper loading states and error messages if the quote generation fails.

2. Commission Quote API Spec

- **Contract:** It must accept and return data based on this agreed contract:
 - **Request Payload:** `loanAmount`, `loanTermInMonths`, `riskBand`
 - **Response Payload:** `quoteId`, `commissionRate`, `totalCommission`
- **Security:** The Commission Quote API is strictly secured. It must require an `api-key` header to process the request. Any request without a valid API key should be rejected.
- **Simulation:** To mimic real-world network conditions, your Commission Quote AP must occasionally (randomly) throw an error.

AI Usage Policy

We recognize that modern software engineering involves leveraging AI tools (e.g., GitHub Copilot, ChatGPT, Gemini, Claude). **Using AI is fully allowed and encouraged during this take-home challenge.**

However, we expect the following:

- **Transparency:** Please include a brief section in your `README.md` outlining how you utilized AI during the challenge (e.g., "Used Copilot to generate boilerplate UI components," "Used ChatGPT to write regex/tests," or "Generated the initial Mock API structure with Claude").
- **Comprehension:** You must take full ownership of the code you submit. You should deeply understand how it works, why it was structured that way, and its potential trade-offs.
- **Next Steps:** If successful, the next stage is a collaborative live-coding session where we will expand this exact codebase. You are welcome to use AI collaboratively with us during that session, and we will likely ask you to walk us through the code you (or the AI) wrote.

Expectations & Evaluation Criteria

Must-haves:

- **Solve the core problem correctly:** The web application successfully captures input, securely communicates with the Commission Quote API, and gracefully handles the response (both success and failure).
- **Keep the code readable and structured clearly:** We look for sensible directory structures, clean abstractions, and good separation of concerns.
- **Include core tests:** We don't need 100% coverage, but please include a few unit or integration tests for the core logic.
- **Handle key edge cases:** What happens if the user submits invalid numbers? What happens when the Commission Quote API times out or throws an error?
- **Make sensible code decisions:** Treat this as the foundation of a real app. Choose a tech stack you are most comfortable with.
- **Include run instructions:** Provide a `README.md` with clear, step-by-step instructions on how to start the application components and run the tests.

Nice-to-haves:

- **UI Polish:** A clean, accessible, and responsive user interface (using basic CSS or a component library like Material-UI/Tailwind is perfectly fine).

Submission

Please push your code to a public or private GitHub repository and share the link with us. Alternatively, you can send us a zip file containing your project.

We aim to review your submission and get back to you with feedback and next steps within 48 hours.

During our next interview stage, we will use this codebase as a starting point for a collaborative live-coding session where we will introduce new business requirements and expand the application together.

Good luck, and have fun!